Yu-Fong Chen - 242115

Proyecto No. 2

Sea $G = (V,\Sigma,S,P)$ donde el conjunto de producciones P contiene:

S $\rightarrow NP\ VP$
V$P \rightarrow VP\ PP$
V$P \rightarrow V\ NP$
V$P \rightarrow cooks \mid drinks \mid eats \mid cuts$
P$P \rightarrow P\ NP$
N$P \rightarrow Det\ N$
N$P \rightarrow he \mid she$
V $\rightarrow cooks \mid drinks \mid eats \mid cuts$
P $\rightarrow in \mid with$
N $\rightarrow cat \mid dog$
N $\rightarrow beer \mid cake \mid juice \mid meat \mid soup$
N $\rightarrow fork \mid knife \mid oven \mid spoon$
D$et \rightarrow a \mid th$e

Definición de PDA a partir de gramática G

En los autómatas de pila (pushdown automata o PDA), se debe tener en mente lo que es un autómata finito (AFN), limitados a la información de su estado actual y el símbolo consumido durante su simulación. De ahí, agregamos el concepto de tener una memoria que se maneja por sí sola, queriendo decir que esta sabe dónde guardar datos y cómo recuperarlos. Adicionalmente, nos ayudaría que esta memoria recuerde lo último guardado y así, esta memoria toma forma de una pila (stack). Si un AFN es una tupla $M = (Q,\Sigma,q0,F,\delta)$ donde Q es el conjunto de estados, Σ el alfabeto del autómata, $q0$ el estado inicial, F el subconjunto de estados de aceptación; y δ la función de transición, la pila se expresa en la función de transición. δ:Q ×(Σ∪{ε}) ×Γ →2^(Q×Γ∗).

Ahora, el método topdown contruye un PDA que reemplaza no-terminales por el cuerpo de sus producciones en la pila, y "consume" terminales en la entrada cuando coinciden con lo que tiene la pila. Sea $G = (V,\Sigma,S,P)$ una gramática libre de contexto donde V son los no-terminales, Σ es el alfabeto (terminales), S es el símbolo inicial y P es el conjunto de producciones. Su PDA correspondiente es $NT(G) = (Q,\Sigma,\Gamma,q0,Z0,F,\delta)$ tal que:

- Q = {$q0,q1,q2$}

- F = {q2}
- Γ =$V \cup \Sigma \cup \{Z0\}$
- La función de transición cumple con:
    - El inicio y final de una derivación por la izquierda:
        - Para colocar el símbolo inicial de la gramática, siempre un no-terminal, en la pila: $\delta(q0,\varepsilon,Z0) = \{(q0,SZ0)\}$
        - El estado $q2$ es el único estado de aceptación, y se alcanza cuando la pila del PDA queda vacía. $\delta(q1,\varepsilon,Z0) = \{(q2,Z0)\}$
    - Operación del PDA:
        - Cada no-terminal encontrado a la cabeza de la pila es reemplazado por los cuerpos de sus producciones.
          $\forall A \in V, \delta(q1,\varepsilon,A) = \{(q1,\alpha) | (A \rightarrow \alpha) \in P\}$
        - Cada terminal encontrado a la cabeza de la pila debe coincidir con el siguiente símbolo de entrada, y cuando lo hace simplemente se extrae de la pila.
          $\forall a \in \Sigma, \delta(q1,a,a) = \{(q1,\varepsilon)\}$
        - Si no coinciden, la simulación no acepta la entrada.

Toda la información/definición del PDA la saqué de "Autómatas de pila 2026".

Investigación y/o cálculo de tasas de crecimiento

Como vamos a estar implementando el algoritmo de Cocke-Tounger-Kasami (CYK) y los PDA para el reconocimiento de una cadena de longitud n, en adelante veremos las tasas de crecimiento.

CYK

El algoritmo funciona en que para una cadena de longitud N , se construye una tabla T de tamaño N x N. Cada celda de la tabla T[i, j] es el conjunto de todos los constituyentes que pueden producir la subcadena que va desde la posición i hasta la j. El proceso consiste en llenar la tabla con las soluciones a los subproblemas encontrados en el análisis sintáctico ascendente. Por lo tanto, las celdas se llenarán de izquierda a derecha y de abajo hacia arriba.

	1	2	3	**4**	**5**
1	[1, 1]	[1, 2]	[1, 3]	[1, 4]	[1, 5]
2		[2, 2]	[2, 3]	[2, 4]	[2, 5]
3			[3, 3]	[3, 4]	[3, 5]
4				[4, 4]	[4, 5]

5					[5, 5]

(Geeks for Geeks, 2025)

Para una gramática fija, el algoritmo recorre las longitudes l = 2,...,n, las posiciones de inicio (n-l+1) y los cortes (l-1). El número de combinaciones de intervalos y cortes es $\Sigma(l-1)(n-l+1) = (n^3-n)/6$. Las comprobaciones de producciones cuestan una constante para esta gramática fija; por lo tanto, el peor caso es $O(n^3)$ y el espacio de tabla es $O(n^2)$, sin contar la representación de árboles.

PDA

La construcción no determinista no garantiza un número polinómico de configuraciones para una gramática arbitraria. Esta implementación enumera configuraciones con BFS; para G fija y entrada de longitud n, la pila útil se limita a n símbolos. El número de pilas posibles está acotado por una suma geométrica de $|\Gamma|^k$ ($k \leq n$), multiplicada por n+1 posiciones, de modo que el peor caso de exploración está acotado por $O(n \cdot |\Gamma|^n)$ configuraciones, y el costo de copiar/inspeccionar pilas añade hasta O(n) por transición: $O(n^2 \cdot |\Gamma|^n)$ en una implementación directa. Esta es una cota superior conservadora, no una afirmación de que toda entrada tarde exponencialmente. La longitud del camino aceptante puede ser mucho menor (dilli_hangrae, 2024).

Discusión de desempeños

En el peor de los casos, CYK tiene una cota polinómimca de $O(n^3)$ y necesita $O(n^2)$ celdas. Tiene cierta mecanisidad en su proceso que hace que uno pueda depender en su proceso, haciendo recursión izquierda o tomando de frente la ambigüedad. Lo que sí, siempre necesita construir su tabla, incluso si la frase que se está analizando no es larga o compleja. El PDA ya es más arraizado a lo que hemos visto la mayoría de la clase, vinculado a las expresiones o reglas de aceptación. Esta sí se adapta a la longitud de las cadenas analizadas, tomándose menos tiempo si es más corta y más, si no. Ahora, el proceso es menos sistemático y explorar sus posibilidades se puede poner minuscioso. CYK es como el proceso largo que siempre da una respuesta y PDA es como la intuición en que, hay que chequearlo un poco más, aunque parezca más simple.

Programas

- PDA de gramática G programado y funcional
- Algoritmo CYK programado y funcional

Link al repositorio: https://github.com/yufyuf02/proyecto02_cc2019.git

Link al video: https://youtu.be/Mpy5YXJBH2M

Referencias

Gálvez P., T. (2026). Autómatas de pila [Material de clase, Teoría de la Computación, CC2019]. Universidad del Valle de Guatemala.

GeeksforGeeks. (s. f.). Cocke–Younger–Kasami (CYK) algorithm. https://www.geeksforgeeks.org/compiler-design/cocke-younger-kasami-cyk-algorithm/

dilli_hangrae. (2024, 7 de mayo). Push down automata (PDA). Medium. https://medium.com/@dillihangrae/push-down-automata-pda-0ef99461fc30